PROJECT :: Java based TCP Chat with Threading

- Due Oct 27 by 11:59pm
- Points 200
- Submitting a file upload
- File Types java
- Available until Oct 29 at 11:59pm

This assignment was locked Oct 29 at 11:59pm.

Release History

You are always responsible for the latest release of assignments. The release may be updated at any time to fix bugs or add clarification. Canvas will send out an announcement if and when the assignment is updated. Make sure that you are receiving these Canvas notifications.

V002 : Clarified shutdown procedure, added Ctrl-C handler code, and corrected output with thread name in lieu of socket file descriptor, added additional helpful notes.
V001 : Initial release, expect a few bugs

Introduction

The objective is to create a chat application where multiple clients can connect to a server and exchange messages in real-time. The server will manage connections, relay messages between clients, and handle join/leave notifications. The project can be divided into two main parts: the server and the client.

Learning Goals

- Understand TCP sockets and how to use them for network communication.
- Implement client-server architecture.
- Manage multiple connections on the server.
- Practice string manipulation and protocol design for message formatting.

Tools and Languages

- **Language**: Java
- **Libraries**: Only what comes with the SDK, you must use Threads

Project Specifications

Server

1. **Listen for Incoming Connections**: The server should be able to accept connections from multiple clients simultaneously.
2. **Broadcast Messages**: When a client sends a message to the server, the server should relay this message to all connected clients.

3. **Handle Joining and Leaving**: Notify all clients when a new client joins or an existing client leaves the chat.
4. **Concurrency**: Manage multiple clients simultaneously. This could be achieved through threading (each client connection is handled by a separate thread) or select/poll mechanisms for a more advanced approach.

Client

1. **Connect to Server**: The client should prompt the user for the server's IP address and port number, then use this information to establish a connection.
2. **Send and Receive Messages**: Allow the user to type messages and send them to the server. Also, display messages from other users.

Handling Multiple Clients

To handle multiple clients in this project, you are required to use threads. Each client connection will be managed by a separate thread, allowing the server to handle simultaneous communication with multiple clients efficiently. Here is an overview of how threading works in this context.

Understanding Threads in Network Programming

Introduction

In network programming, managing multiple client connections to a server can be challenging. Using threads is a common approach that simplifies handling simultaneous client connections by dedicating a separate thread to each client. This allows concurrent communication without needing complex I/O multiplexing mechanisms like `select()`.

How Threading Works in Java

In Java, threads can be used to manage client connections by creating a new thread whenever a client connects to the server. Each thread handles the communication between the server and its specific client, allowing the server to manage multiple clients simultaneously without blocking other connections. The server's main thread remains focused on accepting new connections, while each client thread manages reading and writing operations.

The primary components involved in using threads for this project are:

- **Thread Class:** Java provides a `Thread` class that can be extended or a `Runnable` interface that can be implemented to define the behavior of each client connection.
  - Note: For this project you will use the Runnable interface as this is the preferred modern way to implement threading in Java
- **Synchronization:** If shared resources (e.g., a list of active clients) are accessed by multiple threads, synchronization mechanisms like `synchronized` blocks are needed to ensure thread safety.
- **Thread Lifecycle:** Each client thread will run as long as the client is connected and can be terminated when the client disconnects.

Advantages of Using Threads

- **Simplicity:** Using threads can simplify server design by isolating client handling into separate execution paths, avoiding the complexity of managing I/O readiness.
- **Responsiveness:** The server remains responsive to new connections, as the main thread is not blocked by client communication.
- **Familiarity:** Java's thread model is widely used and provides built-in methods for managing thread lifecycle, making it suitable for beginners in network programming.

Challenges

- **Resource Management:** Each thread consumes system resources. For a large number of clients, this can become a concern.
- **Debugging:** While simpler than using `select()`, multi-threaded programs still need careful handling to avoid issues like race conditions or deadlocks when accessing shared resources.

Conclusion
Using threads for managing client connections is a practical approach for building a chat server in Java. It allows concurrent communication between clients while keeping the server design straightforward. Understanding thread management and synchronization is key to successfully implementing a multi-threaded chat server, providing a solid foundation for network programming in Java.

For details on programming with select(), see **chapter 12 of the David Eck Java book. (https://math.hws.edu/javanotes/c12/index.html)**

Part 1 :: TCP Chat Client

In this part, you will develop a TCP-based chat application using Java. The application will consist of two parts: a client and a server. The server will manage connections from multiple clients and relay messages between them, enabling a simple chat room functionality where all connected clients can communicate with each other in real-time. The client must do hostname lookup so that it is not necessary to supply know the IP address of the server.

Your task will be to implement the client-side of the application.

The provided scaffolding outlines the structure of the client, including function prototypes and descriptions. You will need to complete the implementation of each function based on the provided specifications.

Client Application Overview

The client application allows users to connect to a chat server, send messages, and receive real-time updates from other users. It involves setting up a TCP connection to the server, handling user input, and processing incoming messages from the server. Below is a detailed breakdown of what each part of the client application needs to achieve:

Port Selection

All clients should use port **12345** for connecting to the server. This ensures that all students' applications are compatible with each other and can connect to the same server instance during testing. Port 12345 is chosen as it is not typically reserved by other services, making it suitable for this project.

Outline of the Client Application:

1. **Establishing Connection**
   - **Initialize Connection:** The client prompts the user for the server's IP address and port number. It then attempts to establish a TCP connection with the chat server.
   - Use Java's `Socket` class to create a connection to the specified server. If the connection is successful, proceed to the next steps. Otherwise, handle connection errors gracefully.
2. **Receiving Server Messages**
   - **Handle Server Messages:** Continuously listen for messages from the server. When a message is received, display it to the user.
   - Implement this using a dedicated thread that reads from the `InputStream` of the `Socket`. This thread should:
     - Read incoming messages from the server.
     - Ensure each message is correctly formatted and then print it to the console.
     - Handle connection loss or server shutdown by notifying the user and gracefully exiting the thread.
3. **Sending User Input**
   - **Handle User Input:** Allow the user to type messages and send them to the server.
   - Use another thread to handle user input:
     - Read input lines from the console.
     - Prepend the user's username to the message for identification.
     - Send the formatted message to the server using the `OutputStream` of the `Socket`.
     - Handle special commands (e.g., "/quit") to allow users to exit the chat gracefully.
4. **Client Workflow**
   - **Main Method Steps:**
     - Validate Command-Line Arguments: Ensure that the user provides the server's hostname and their username as arguments.
     - Create a `Socket` to connect to the server using the provided hostname and port.
     - Start a thread for receiving messages from the server.
       - Note: I suggest that you first the get the client working without threads as it is simpler. Use the example from the slides to get started.
     - Start another thread for handling user input.

- Keep the main thread running to allow the input and message threads to operate concurrently.
- When the user decides to quit or the server disconnects, cleanly close the socket and terminate all threads.

5. **Clean Up**
    - **Shutdown Handling:** Ensure that when the client application is terminating, it closes the socket and stops any running threads.
    - Handle exceptions and edge cases, such as the server disconnecting unexpectedly, and notify the user appropriately.

Documentation on Command-Line Arguments

When running the chat client, you should provide two command-line arguments in addition to the executable name:

1. **Server's Hostname or IP Address**: This is the address where the chat server is running. It can be an IP address (e.g., `192.168.1.10`) or a hostname (e.g., `localhost`) if the server is running on the same machine as the client.
2. **Username**: This is the identifier that will be used when sending messages. It allows other users in the chat to recognize who the message is from. The username will be prepended to your messages automatically by the client before sending them to the server.

Hence, the way to run your client from the terminal would look like this:

`java ./tcpccs.java <Server's Hostname> <Username>`

For example, the following will initiate the client, attempt to connect to the chat server running on `localhost`, and use "Alice" as the username for the session.

java `./tcpccs.java localhost Alice`

Part 2 :: TCP Chat Server

For the server part of your TCP chat application, you will design and implement the server-side logic to handle multiple client connections, receive messages from any connected client, and broadcast these messages to all other connected clients. This will enable real-time communication between clients connected to your server.

Server Application Overview

The server manages client connections, relays messages between clients, and handles client joining and leaving notifications. It uses threads to allow concurrent communication with multiple clients. Below is a detailed breakdown of what each part of the server needs to achieve:

Outline of the Server Application:

1. **Initializing the Server**
   - **Setup Server Socket:** Create a server socket that listens on port **12345**. This involves:
     - Creating a `ServerSocket` object.
     - Binding it to the specified port.
     - Setting it to listen for incoming client connections.
   - Ensure that the server is ready to accept client connections before proceeding.
2. **Accepting New Connections**
   - **Manage Client Connections:** Accept connections from multiple clients and create a new thread for each client.
     - When a new client connects, accept the connection using the `accept()` method of `ServerSocket`.
     - Create a new `Thread` to handle communication with this client.
     - Keep track of active client connections using a collection (e.g., `List<Socket>` or `ConcurrentHashMap`).
       - **Concurrency and Collections:**
         When managing multiple client connections with threads, using non-concurrent collections can cause issues like race conditions. To avoid these problems, it's best to use thread-safe collections like `ConcurrentHashMap`, which automatically handles concurrent access. This choice simplifies code and reduces the need for manual synchronization, making it the preferred option for managing shared data between threads.
3. **Handling Client Communication**
   - **Client Interaction:** Each client thread handles sending and receiving messages for a particular client.
     - Read incoming messages from the client using the `InputStream` of the `Socket`.
     - When a message is received, broadcast it to all other connected clients.
     - If a client disconnects, remove the client from the list of active connections and notify other clients if necessary.
4. **Broadcasting Messages**
   - **Send Messages to Clients:** Relay messages received from a client to all other connected clients.
     - For each message received, iterate through the list of active clients.
     - Send the message using the `OutputStream` of each client's socket, ensuring that the sender does not receive their own message.
     - Ensure that messages are properly formatted for readability.
5. **Server Workflow**
   - **Main Method Steps:**

- Initialize the server by creating and setting up the `ServerSocket`.
- Accept new client connections and start a new thread for each connection.
- Continuously listen for client activity using the main server thread.
- Use client threads to handle message reading and writing concurrently.
- If a client disconnects, properly clean up the connection and remove it from the active list.
  - ***Note:*** It may be easier, and certainly acceptable, to just mark the client handler object as inactive in lieu of actually removing it from the list. You should make sure that you are checking for this when broadcasting. In other words, you will only broadcast to active clients.

- **Printing the Thread name**
  - Java assigns a thread name to each thread. Thread ID is deprecated. You can get the default thread name with the following line of code:

    `Thread.currentThread().getName())`

6. **Clean Up**

- **Shutdown Handling:** When the ***server*** shuts down, close the server socket.
  - While it's a good idea to shutdown all client connections, it adds too much complexity at this stage. We will expore this in more detail in a later assignment.

    You may use the following code to capture Ctrl-C on the server:

    ```
    boolean running = true;
    ```

    ```
    // Register shutdown hook
    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
      System.out.println("Shutdown signal received.");
      running = false;  // Set running to false to stop the server loop
    }));
    ```

Command-Line Arguments

The server does not require any command-line arguments for its operation. It is designed to bind and listen on a predefined port (`PORT`). However, you should consider implementing functionality to allow the port number to be specified via command-line arguments as an optional feature.

Typical Execution

Your output should look exactly like the following. There are no tabs embedded in this so where you see a space it is an ASCII space character. I will try to be as gracious as possible while testing, however, the closer you are to this, the more likely it is that you will earn all of the points.

Server:

```
$ java ./tcpcss.java
Listener on port 12345
Waiting for connections ...
New connection, thread name is Thread-0, ip is: 127.0.0.1, port: 35952
Adding to list of sockets as 0
New connection, thread name is Thread-1, ip is: 127.0.0.1, port: 44770
Adding to list of sockets as 1
client: bob: hi all
client: sally: hi bob
client: bob: hi there sally, how's it going?
client: sally: pretty good bob, how about you?
Host disconnected, ip 127.0.0.1, port 44770
Host disconnected, ip 127.0.0.1, port 35952
```

Client 1 [bob]:

```
$ java ./tcpccs.java localhost bob
Connected to the server. You can start sending messages.
hi all
sally: hi bob
hi there sally, how's it going?
sally: pretty good bob, how about you?
^C
```

Client 2 [sally]:

```
$ java ./tcpccs.java localhost sally
Connected to the server. You can start sending messages.
bob: hi all
hi bob
bob: hi there sally, how's it going?
pretty good bob, how about you?
^C
```

Bad Command Line arguments:

```
$ java ./tcpccs.java
Usage: ./tcpccs <server_hostname> <username>
```

While your code should report as many errors as possible, and we may require this in the next assignment so being careful now will save you time then, no other error checking will be tested for this assignment.

Deliverables and Evaluation

Similar to the client part of this assignment, you will submit the source code. Your files must be named tcpcss`.java` (server) and tcpccs`.java (client).` Upload your files here to Canvas. Don't worry about file renaming, we know that it does that the autograder can handle it.